Reconocimiento de señas aisladas de la Lengua de Señas Colombiana (LSC-54): plan experimental, Entrega 1

Andrés Díaz, Lorena Valero, Jorge Anaya y Sabrina Miranda
Maestría en Analítica Aplicada e Inteligencia Artificial, Universidad de La Sabana
Técnicas Avanzadas de Modelado, 2026-II · Octubre de 2026
https://github.com/andresdila/lsc54-reconocimiento-senas

Resumen. Este documento presenta el plan experimental para clasificar 43 señas aisladas de la LSC a partir de landmarks de MediaPipe, evaluando siempre sobre señantes que no participaron en el entrenamiento. Se describen el contexto, el origen y la licencia de los datos, su auditoría inicial, la partición por señante, los riesgos de sesgo, la línea base, las métricas de éxito, la arquitectura candidata, el cronograma y los criterios de reproducibilidad.

1. Contexto y alcance del problema

La lengua de señas colombiana (LSC) es la lengua con la que se comunica la comunidad sorda del país, y el acceso a su aprendizaje formal sigue siendo limitado. Precisamente por esta razón, el conjunto de datos LSC-54 fue construido para respaldar una herramienta de enseñanza de vocabulario básico (Mora-Zarate et al., 2025). El proyecto toma ese mismo propósito como contexto de aplicación: un estudiante que practica frente a una cámara y recibe retroalimentación sobre si la seña que ejecutó corresponde a la que se le pidió. Este escenario define la unidad de generalización del experimento, pues el sistema siempre se enfrentará a *una persona nueva*, que no participó en el entrenamiento.

El problema se plantea como una clasificación multiclase de señas aisladas: dado un video corto, se debe asignar una de 43 etiquetas de vocabulario (33 expresiones de cortesía y los números del uno al diez). Se ubica entre las series de tiempo y la visión por computador, porque cada video se convierte, mediante MediaPipe (Lugaresi et al., 2019), en una serie multivariada de coordenadas de manos y torso, y es esa serie, no los píxeles, la que recibe el modelo. A partir de ello, el alcance se delimita de manera explícita:

- **Dentro del alcance:** 43 clases de la campaña S2 del dataset, señas aisladas, evaluación independiente del señante, entrada en forma de landmarks y comparación entre un modelo clásico y uno secuencial.
- **Fuera del alcance:** lengua de señas continua, traducción, la campaña de colores (S1), reconocimiento en tiempo real y cualquier despliegue con usuarios sordos sin su validación previa.

Pregunta experimental. Bajo una partición independiente del señante, ¿un modelo secuencial (LSTM/GRU) sobre landmarks de MediaPipe mejora el macro-F1 frente a un baseline clásico sobre estadísticas de los landmarks, y a qué costo computacional? La hipótesis de trabajo es que la dinámica temporal del movimiento aporta información que los resúmenes estadísticos pierden; sin embargo, con apenas 14 señantes de desarrollo no es obvio que el modelo secuencial supere al clásico, y por eso la comparación se diseña para poder rechazar la hipótesis con la misma claridad con que se podría aceptar.

2. Origen y características del dataset

LSC-54 fue publicado por Mora-Zarate, Garzón-Castro y Castellanos-Rivillas (2025) en *Data in Brief* (63:112145), con DOI del conjunto de datos 10.57760/sciencedb.25639 en Science Data Bank y licencia **CC BY-NC 4.0**, es decir, uso académico y no comercial con atribución. Por esta razón los videos no se redistribuyen en el repositorio del proyecto; el script *download_data.py* los obtiene de la fuente oficial y verifica su MD5. Los autores reportan 54 señas (11 colores, 33 de cortesía y 10 números), 22 participantes y entre tres y cinco repeticiones por seña. Además, publican una versión aumentada ×45 en un archivo de 51,4 GB que **no** se usa, porque multiplicar los mismos videos antes de particionar es una fuente clásica de fuga de información (Scikit-learn Developers, s. f.).

Con base en el inventario versionado (*data/inventory/*), la composición del subconjunto de trabajo es la siguiente.

Característica	**Valor en el subconjunto S2 (después de excluir duplicados)**
Unidad y formato	Un video *.avi* por repetición de una seña; 1.740 videos, 671 MB (mediana 290 KB por video, rango 80 KB a 2,0 MB)
Clases	43: 33 de cortesía (1.198 videos) y 10 números (542 videos)
Señantes	18 carpetas: 9 graban las 43 clases (*full*) y 9 graban 21 (11 de cortesía y 10 números, *partial*)
Videos por clase	26 a 57. Las 21 clases comunes tienen entre 51 y 57; las 22 restantes solo las graban 9 señantes y tienen entre 26 y 30
Repeticiones	1.710 videos con índice de repetición 0 a 2; 30 con índice 3 a 5
Resolución y velocidad	30 fps en todos; 640×360 en la mayoría y 640×480 en S2-21, S2-24 y S2-25 (el artículo reporta 1080p a 30 fps)

Cuadro 1: Composición del subconjunto de trabajo (cifras calculadas del inventario versionado).

En cuanto a las limitaciones, es necesario señalarlas con la misma claridad con la que se describen las ventajas, siguiendo la lógica de las hojas de datos propuestas por Gebru et al. (2021). Primero, el material disponible no documenta edad, sexo, tono de piel ni mano dominante de los señantes, y tampoco precisa si son personas sordas, oyentes o intérpretes. Segundo, el conjunto contiene 38 identidades de señante (20 en S1 y 18 en S2, contadas por carpeta) frente a 22 participantes reportados, de modo que la correspondencia entre carpetas y personas no es verificable y algunas carpetas de campañas distintas probablemente son la misma persona. Tercero, el artículo reporta una grabación a 1080p y 30 cuadros por segundo, pero los videos publicados están reducidos: todos a 30 fps, la mayoría a 640×360 y tres señantes (S2-21, S2-24 y S2-25) a 640×480. Esta heterogeneidad de resolución afecta el preprocesamiento (sección 5). Por último, el vocabulario es pequeño y de señas aisladas, lo cual limita cualquier afirmación sobre la lengua en uso real.

3. Auditoría inicial de calidad

La auditoría se ejecuta con *download_data.py*, que deja como evidencia versionada el inventario completo (sha256 por video) y un resumen en *lsc54_v4_inventory.audit.json*. A continuación se presentan sus hallazgos sobre los videos y sobre la detección de landmarks.

Videos. Los tres archivos comprimidos se validan por MD5. El inventario registra 2.880 videos en total, de los cuales se excluyen 30 que corresponden a una copia de la carpeta de números dentro de *Videos Courtesy.zip* (29 de 30 archivos idénticos por CRC32 y tamaño; se conserva *Videos Numbers.zip* como fuente canónica). Tras la exclusión quedan 2.850 videos, 54 clases y 38 identidades de señante (20 en S1 y 18 en S2), sin duplicados exactos por sha256 y con identificadores únicos. Así mismo, tres carpetas con nombres mal escritos se normalizan de forma explícita en la configuración (*dies → diez*, *denada → de_nada*, *maso_menos → mas_o_menos*), de modo que la corrección queda auditada y no oculta en el código. Los hallazgos estructurales más relevantes para el diseño son el desbalance entre clases (relación de 2,2 a 1 entre la más y la menos frecuente) y la cobertura desigual de señantes por clase (9 frente a 18).

Detección de landmarks. La extracción procesó los 1.740 videos (113.658 cuadros) y registró por video, en *landmarks_audit.csv*, resolución, cuadros por segundo, número de cuadros y tasa de detección de pose y de cada mano. Ningún video quedó vacío y la pose se detecta en el 99,5 % de los cuadros; en cambio, la mano más activa aparece en promedio en el 60 % de ellos, posiblemente porque las manos en reposo no se detectan. El 32 % de los videos queda bajo el umbral del 50 %; excluirlos eliminaría un tercio de los datos, por lo que se conservan y la tasa de detección se mantiene como variable y como eje del análisis de errores. La detección varía de forma marcada entre señantes,

de 0,46 (S2-19) a 0,97 (S2-32), y es mayor en los cuatro señantes del test (0,73) que en dev (0,57); por tanto, el desempeño sobre el test podría resultar optimista, y así se reportará.

4. Esquema de partición

La partición v1 se decidió y se versionó *antes* de extraer landmarks o entrenar cualquier modelo, y se hace sobre señantes, nunca sobre videos. La razón es que las repeticiones de una misma persona se parecen mucho entre sí, de modo que una división aleatoria por video pondría a la misma persona a ambos lados y mediría memorización de la identidad en lugar de generalización a personas nuevas (Scikit-learn Developers, s. f.; Zinkevich, s. f.). Por la misma razón se excluyó S1: su numeración de carpetas es independiente de la de S2, y no hay forma de saber si una carpeta de S1 y una de S2 son la misma persona.

Partición	**Señantes**	**Videos**	**Composición**
Dev (folds 1 a 7)	14	1.359	7 folds de 192 a 206 videos; cada fold valida con un señante *full* y uno *partial*
Test bloqueado	4	381	S2-01, S2-03 (*full*), S2-29 y S2-31 (*partial*); semilla 2026

Cuadro 2: Partición v1 por señante.

El test bloqueado reúne 381 videos (21,9 %) y cada fold de validación tiene entre 192 y 206. La estratificación por tipo de cobertura (*full* o *partial*) garantiza que las 43 clases estén presentes en el test y en cada fold; sin ella, un fold podría quedar sin las 22 clases que solo graban los señantes *full*. Esta decisión tiene una consecuencia que debe tenerse presente: en cada fold esas 22 clases son validadas por un único señante (unos tres videos) y, en el test, por dos (6 a 13 videos por clase). Por tanto, el macro-F1 por fold es ruidoso, y se reportará también sobre las predicciones fuera de muestra de los siete folds agrupadas.

Las medidas anti-fuga son las siguientes: (i) *tests/test_split.py* verifica que ningún señante aparezca en dos particiones, que todas las clases estén en test y en cada fold, y que el split versionado se reproduzca exactamente; incluye además una prueba que planta una fuga y comprueba que se detecta; (ii) *make_split.py --check-only* se niega a sobrescribir una partición existente; (iii) imputación, estandarización y cualquier ajuste dependiente de los datos se ejecutan *dentro* de cada fold; (iv) la selección de hiperparámetros se hace solo con dev; y (v) el test se evalúa una única vez, y el script de la línea base se niega a repetirlo.

5. Preprocesamiento

Cada video pasa por MediaPipe Holistic (versión 0.10.21, la misma de los autores) y de cada cuadro se conservan las coordenadas (x, y) de siete puntos de pose del tren superior (nariz, hombros, codos y muñecas) y de los 21 puntos de cada mano, es decir, 98 valores por cuadro. No se usan landmarks faciales, tanto por parsimonia como para reducir la identificabilidad de las personas. Un detalle importante es que MediaPipe entrega x normalizada por el ancho e y por el alto de la imagen, de modo que con dos resoluciones distintas (640×360 y 640×480) la forma de una misma seña queda deformada de manera diferente según el señante. Por esta razón las coordenadas se convierten a píxeles (x·ancho, y·alto) antes de normalizar; de lo contrario, el modelo podría aprender *qué cámara* se usó en lugar de *qué seña* se hizo. Después, las coordenadas se normalizan por cuadro tomando como origen el punto medio de los hombros y como escala la distancia entre ellos, lo que vuelve la representación invariante a la posición y a la distancia a la cámara. Cuando una mano no se detecta, sus valores quedan como ausentes y la tasa de detección se conserva como información, en vez de interpolarse en silencio.

Para el baseline, cada video se resume en 493 variables: media, desviación, mínimo, máximo y velocidad media absoluta de cada coordenada, la tasa de detección de cada mano y el número de cuadros. Para el modelo secuencial, la serie se remuestrea a 32 cuadros, se completan los vacíos cortos

(hasta 5 cuadros) por interpolación lineal, se añade un canal de máscara para los vacíos restantes y se aplican aumentos solo sobre entrenamiento (desplazamiento temporal, escala y rotación pequeñas, ruido gaussiano). No se emplea reflejo horizontal, porque cambiaría la lateralidad de la seña.

6. Riesgos de sesgo, privacidad y uso indebido

Para identificar y ordenar los riesgos se toman como guía las funciones *Map*, *Measure* y *Manage* del marco de gestión de riesgos de IA del NIST (2023). El Cuadro 3 resume los más relevantes con la evidencia disponible y la mitigación prevista.

Riesgo	Evidencia	Mitigación
Representatividad de los señantes	18 personas sin metadatos demográficos; no se sabe si son sordas, oyentes o intérpretes	Reportar el desempeño por señante (peor caso y dispersión), no solo el promedio; limitar las conclusiones a este grupo
Fuga de identidad	38 identidades por carpeta (20 en S1, 18 en S2) frente a 22 participantes	Excluir S1; unidad de partición = señante; controles automáticos
Detección desigual de MediaPipe	Puede variar con la iluminación, la ropa, las oclusiones y las características de la persona	Auditar la tasa de detección por señante y relacionarla con el error del modelo
Atajos no lingüísticos	La duración, la tasa de detección y la resolución (640×360 frente a 640×480) pueden identificar al señante o a la cámara más que a la seña	Convertir a píxeles antes de normalizar; ablación sin *n_frames* ni tasas de detección; revisión de errores por señante
Desbalance y cobertura	26 a 57 videos por clase; 22 clases con solo 9 señantes	Macro-F1 como métrica principal; pesos de clase como ablación; reporte por clase
Privacidad	Los videos publicados tienen los rostros pixelados por los autores y están bajo CC BY-NC	No redistribuir videos; versionar solo landmarks e inventario; sin landmarks faciales
Uso indebido o sobreafirmación	Señas aisladas no equivalen a la lengua; un error afecta a quien aprende	No presentarlo como traductor ni como sustituto de intérpretes; validar con la comunidad sorda y docentes de LSC antes de cualquier uso real

Cuadro 3: Riesgos identificados, evidencia y mitigación.

7. Línea base implementada y primeras métricas

Siguiendo la recomendación de comenzar con un modelo sencillo y bien medido antes de complicar el sistema (Zinkevich, s. f.), la línea base consiste en un *Pipeline* de scikit-learn con imputación por mediana, estandarización y regresión logística multinomial (C = 1, semilla 2026) sobre las 493 variables descritas. El código está en el repositorio: *src/lsc/features/landmarks.py* (representación y estadísticas), *scripts/extract_landmarks.py* (extracción y auditoría de detección) y *scripts/train_baseline.py* (validación cruzada agrupada con los folds de la partición v1). Las pruebas *tests/test_features.py* verifican la invarianza a traslación, escala y resolución, y el manejo de manos ausentes. Además del modelo clásico, el mismo script calcula dos referencias triviales que fijan el piso que cualquier modelo debe superar.

Las cifras completas están en *reports/e1_plan/*; el test bloqueado no se ha evaluado. La regresión logística alcanza un macro-F1 de 0,549, casi treinta veces el de la referencia aleatoria, y una exactitud top-3 de 0,782. Su costo es bajo (1,8 s de ajuste por fold y 0,03 ms de predicción por video), mientras

Modelo (validación cruzada, 7 folds)	Macro-F1 (media ± d.e.)	Macro-F1 agrupado	Exactitud (media ± d.e.)
Clase mayoritaria	0,001 ± 0,000	0,003	0,032 ± 0,001
Aleatorio según frecuencias	0,019 ± 0,008	0,020	0,022 ± 0,008
Regresión logística sobre landmarks	0,549 ± 0,167	0,573	0,569 ± 0,150

Cuadro 4: Línea base en dev (validación cruzada por señante). El azar puro sobre 43 clases da una exactitud de 1/43 ≈ 0,023.

que la extracción con MediaPipe toma unos 2,5 s por video en un núcleo de CPU y domina la latencia. La dispersión entre folds es alta (0,20 a 0,69) y se concentra en el fold 3, cuyo señante S2-19 obtiene 0,11; descartar los cuadros con coordenadas físicamente implausibles apenas lo eleva a 0,15, de modo que la línea base se mantiene tal como fue especificada y la causa se estudiará en el análisis de errores de la Entrega 2.

8. Métricas de éxito

La métrica principal es el **macro-F1**, que promedia el F1 de las 43 clases con el mismo peso. Se elige porque las clases tienen frecuencias distintas y porque las 22 clases con menor cobertura de señantes no deben quedar eclipsadas por las 21 más frecuentes; la exactitud, en cambio, premia al modelo que acierta lo común. Como métricas secundarias se reportan la exactitud, la exactitud top-3 (pertinente en una herramienta de práctica, donde sugerir las señas más probables es útil), el F1 por clase y la matriz de confusión, y la distribución del macro-F1 *por señante*, que es la que más se acerca a la experiencia de una persona nueva. El costo se mide con el número de parámetros, el tiempo de entrenamiento y la latencia de inferencia por video en CPU.

Los criterios de decisión se fijan ahora, antes de ver resultados del modelo secuencial: (i) el modelo candidato se considera mejor que el baseline si su macro-F1 de validación cruzada es al menos 0,05 superior (diferencia absoluta) y mejora en al menos 5 de los 7 folds; (ii) como los folds son pocos, la diferencia se acompaña de un intervalo de confianza por remuestreo de señantes y no se interpreta mediante un valor *p* aislado; (iii) el test bloqueado, con solo 4 señantes, se usa únicamente para confirmar la decisión final y se reporta con la salvedad de su amplia incertidumbre. Estos umbrales son una propuesta del equipo y pueden ajustarse antes de ejecutar el modelo secuencial, no después.

9. Arquitectura candidata

El candidato es una red recurrente bidireccional que recibe la secuencia de 32 cuadros (98 coordenadas más el canal de máscara), cuyo flujo completo es: video → MediaPipe Holistic → normalización y 32 cuadros → proyección → BiGRU → atención temporal → capa densa de 43 clases. Una proyección lineal con normalización de capa lleva cada cuadro a 128 dimensiones; dos capas de GRU bidireccional (Cho et al., 2014) con 128 unidades ocultas procesan la secuencia; un agrupamiento por atención temporal produce un vector por video; y una capa densa con *dropout* de 0,3 entrega las 43 probabilidades. Se entrena con entropía cruzada, AdamW, parada temprana sobre el fold de validación y semilla fija. La LSTM (Hochreiter y Schmidhuber, 1997) se incluye como variante, y como ablaciones se contemplan: sin aumentos, sin máscara, sin *n_frames*, con pesos de clase y una convolución temporal 1D de tamaño comparable, para saber si la recurrencia es realmente lo que aporta.

10. Cronograma y roles

Las tres entregas del curso tienen como plazo el 25 de octubre de 2026, por lo que se fijan cierres internos escalonados; lo completado está respaldado por el historial del repositorio.

Los **roles** del equipo son: Andrés Díaz, datos, inventario y partición; Lorena Valero, modelo secuencial y entrenamiento; Jorge Anaya, línea base, métricas y análisis estadístico; y Sabrina Miranda, riesgos de sesgo, documentación y redacción de informes. Cada cambio de código se

Fase	Actividades	Cierre	Estado
Preparación	Repositorio, entorno, descarga con MD5, inventario y auditoría de duplicados, partición v1 y pruebas anti-fuga	17 oct.	Completada
Entrega 1 (plan)	Extracción de landmarks, auditoría de detección, línea base con regresión logística, este documento	20 oct.	Completada
Entrega 2 (avance)	BiGRU/LSTM, ajuste de hiperparámetros en dev, ablaciones, análisis por señante y por clase	23 oct.	Pendiente
Entrega 3 (final)	Evaluación única sobre el test bloqueado, análisis de errores y de costo, informe, presentación y video	25 oct.	Pendiente

Cuadro 5: Cronograma por hitos.

integra mediante *pull request* con revisión de otra persona del equipo.

11. Criterios de reproducibilidad

Para que cualquier persona pueda repetir el experimento y obtener los mismos resultados se establecen los siguientes criterios. Primero, el entorno es Python 3.11 o 3.12 con dependencias declaradas en *requirements.txt* (MediaPipe fijado en 0.10.21) y, tras la primera instalación en ambos sistemas operativos, un archivo de versiones exactas. Segundo, los datos se obtienen con un solo comando, se verifican por MD5 y su inventario con sha256 se versiona como evidencia. Tercero, las decisiones experimentales viven en archivos YAML (*configs/*) y no en el código, la semilla es 2026 en todos los componentes aleatorios, y la partición v1 es inmutable: cambiarla exige crear la v2 de forma explícita. Cuarto, cada ejecución guarda en un JSON su configuración, el *commit* de Git y las versiones de las librerías, y el seguimiento de experimentos se hace con MLflow. Por último, las pruebas automáticas (*pytest -q*) se ejecutan en cada cambio, y el test bloqueado solo se evalúa una vez, con un script que se niega a repetirlo.

Referencias

Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., y Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv:1406.1078*.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daumé III, H., y Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM, 64*(12), 86-92. https://doi.org/10.1145/3458723

Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780.

Lugaresi, C., et al. (2019). MediaPipe: A framework for building perception pipelines. *arXiv:1906.08172*.

Mora-Zarate, J. E., Garzón-Castro, C. L., y Castellanos-Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief, 63*, 112145. https://doi.org/10.1016/j.dib.2025.112145

National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf

Scikit-learn Developers. (s. f.). Common pitfalls and recommended practices. https://scikit-learn.org/stable/common_pitfalls.html

Zinkevich, M. (s. f.). Rules of machine learning: Best practices for ML engineering. https://developers.google.com/machine-learning/guides/rules-of-ml